

ER Diagrams in DBMS

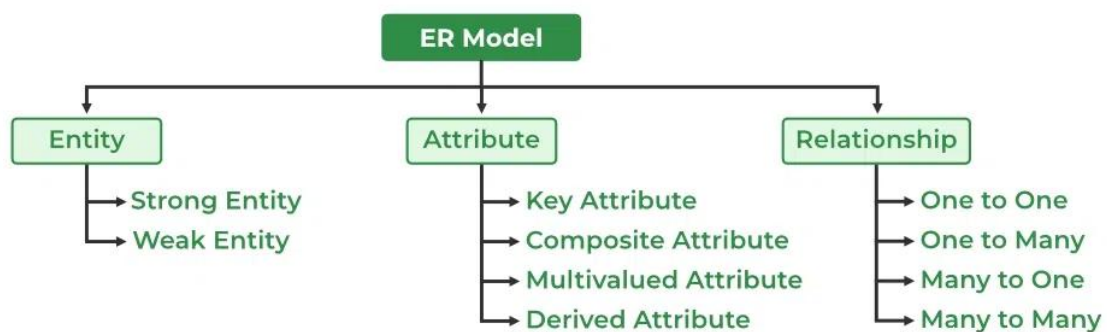
ER model in DBMS is the high-level **data** model. It stands for the Entity-relationship model and is used to represent a logical view of the system from a data perspective. In simple words, the entity relationship diagram is a blueprint that can be used to create a database. E-R diagrams are used to model real-world objects like a person, a car, a company and the relation between these real-world objects.

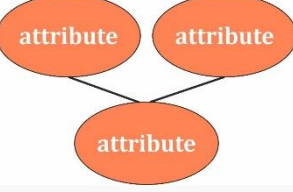
Features of ER model

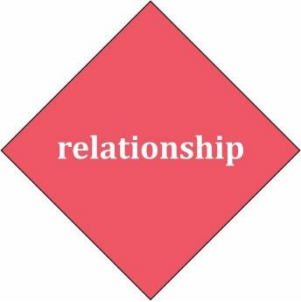
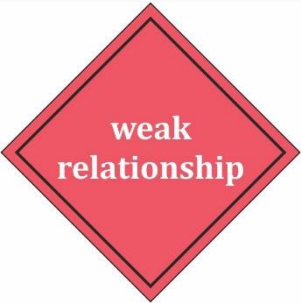
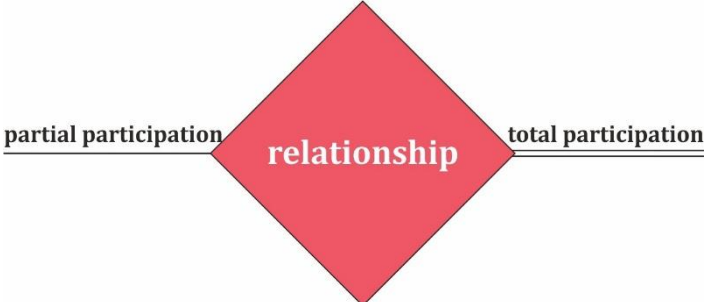
- E-R diagrams are used to represent E-R model in a database, which makes them easy to be converted into relations (tables).
- E-R diagrams provide the purpose of real-world modelling of objects which makes them intensely useful.
- E-R diagrams require no technical knowledge and no hardware support.
- These diagrams are very easy to understand and easy to create even by a naive user.
- It gives a standard solution of visualizing the data logically.

ER diagram basically having three components:

1. **Entities** – It is a real-world thing which can be a person, place, or even a concept. For Example: Department, Admin, Courses, Teachers, Students, Building, etc are some of the entities of a School Management System.
2. **Attributes** – An entity which contains a real-world property called an attribute. For Example: The entity employee has the property like employee id, salary, age, etc.
3. **Relationship** – Relationship tells how two attributes are related. For Example: Employee works for a department.



Component	Symbol
Entity	
Weak Entity	
Attribute	
Key Attribute	
Composite Attribute	
Multivalued Attribute	
Derived Attribute	

Component	Symbol
Relationship	
Weak Relationship	
Participation Constraints	

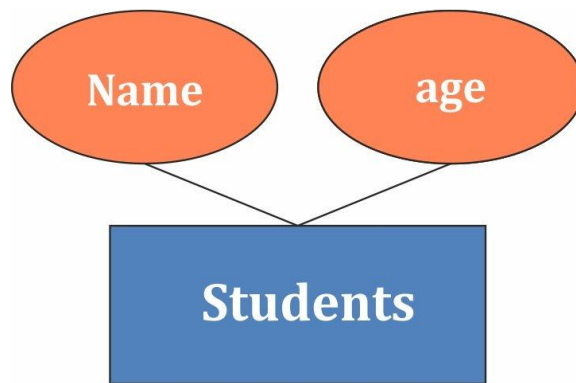
Attributes in DBMS

Attributes define the properties of a **data** object of entity. For example if student is an entity, his ID, name, address, date of birth, class are its attributes.

Types of Attributes in DBMS

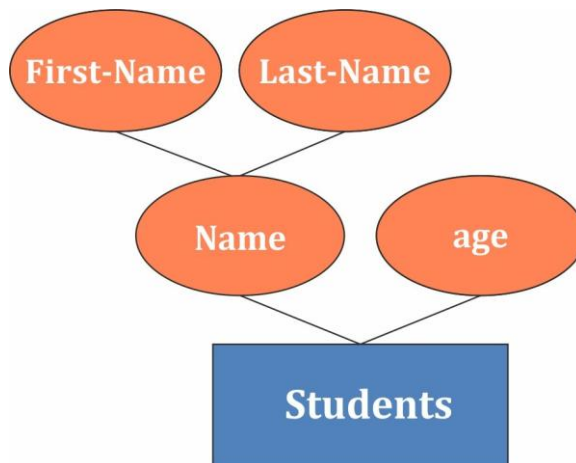
1. Simple Attributes:

Simple attributes are attributes that are drawn from the atomic value domains, which cannot be divided further.



2. Composite attribute:

Composite attributes are those attributes which are composed of many other simple attributes. Composite attributes are made of more than one simple attribute. For example, a student's complete name may have first-name and last-name.

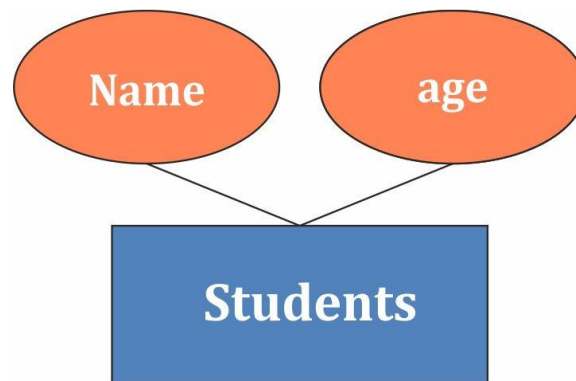


Simple Vs. Composite Attributes

Simple Attribute	Composite Attribute
The attribute which cannot further split into its components is a simple attribute.	The file system provides the details of data representation and storage of data.
Example: The marks of a student, the age of an employee etc.	Example: Name of the student can split into first, middle and last name.

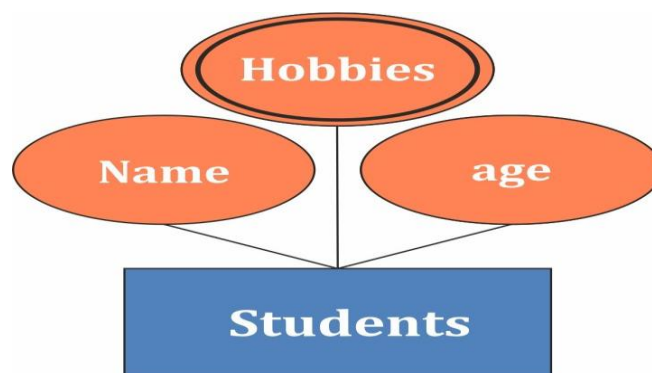
3. Single Valued Attributes:

Single valued attributes are those attributes which can take only one value for a given entity from an entity set. Single-value attributes contain single value. For example – age, gender etc.,



4. Multi Valued Attributes:

Multi valued attributes are those attributes which can take more than one value for a given entity from an entity set. Multi-value attributes may contain more than one values. For example, a person knows more than one Languages, hobbies etc.



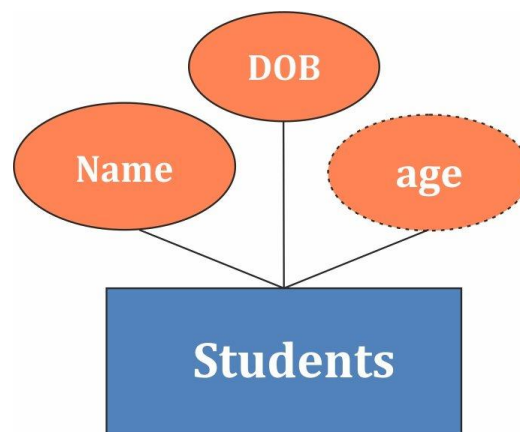
Single-Valued Vs. Multi-Valued Attributes

Single-Valued Attribute	Multi-Valued Attribute
The attribute which has a single value for each entity instance is known as single-valued attribute. There is no alternative of this value.	The attribute which takes up more than one value for each entity instance is known as multi valued attribute.
Example: The RollNo, DOB, Gender of a student will always be a single value.	Example: A person know more than one Languages, and there hobbies

5. Derived Attributes:

Derived attributes are those attributes which can be derived from other attribute(s).

Derived attributes are the attributes that do not exist in the physical database, but their values are derived from other attributes present in the database. For example, age of the student should not be saved directly in the database, instead it can be derived from Date of Birth.



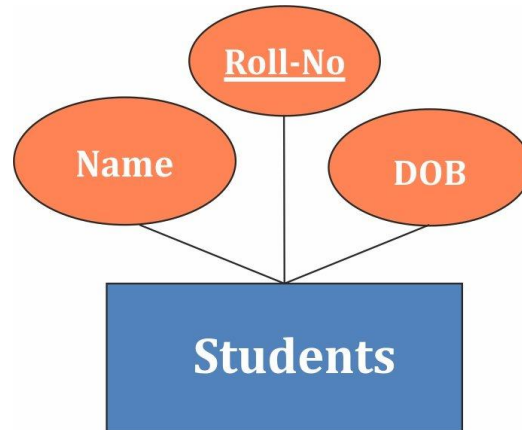
Stored Vs. Derived Attributes

Stored Attribute	Derived Attributes
An attribute which cannot derive from other attributes, and it is mandatory to be stored in the database.	An attribute which is determine the value from other attributes is a derived attribute and it is not mandatory to be stored in the database.
Example: The value which is fixed like Date of Birth.	Example: Age of student can be derived from DOB.

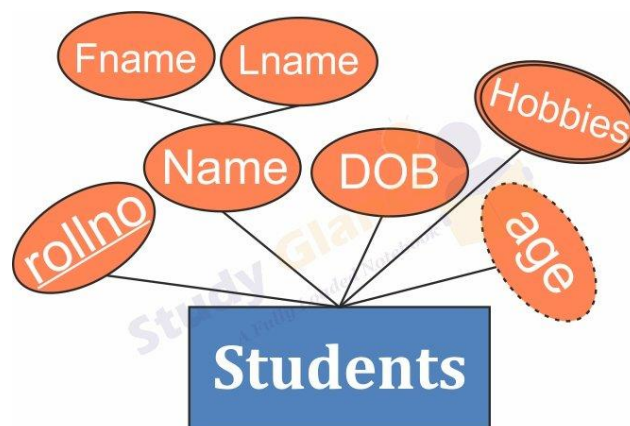
6. Key Attributes:

Key attributes are those attributes which can identify an entity uniquely in an entity set.

Key is an attribute or collection of attributes that uniquely identifies an entity among entity set. For example, the roll_number of a student makes him/her identifiable among students.



Student Entity with all types of Attributes



Entity in DBMS

Entity: An entity is anything in the real world, such as an object, class, person, or place. Objects that physically exist and are logically constructed in the real world are called entities. An entity is distinguishable from other entity.

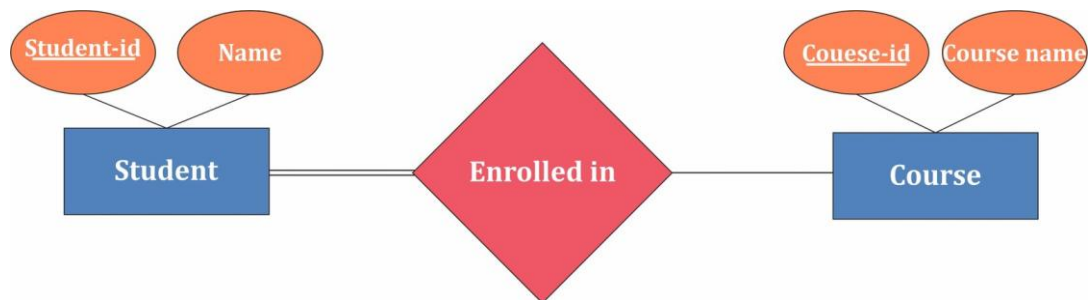
Entity type: The entity type is a collection of the entity having similar attributes.

Entity set: is a group of entities of similar kinds. It can contain entities with attributes that share similar values. It's collectively a group of entities of a similar type. The entity set need not be disjoint.

In summary, an Entity is an object of a Type Entity and the set of all entities is called an entity set.

Entities are of two types:

Strong Entity – A strong entity is an entity type that has a key attribute. It doesn't depend on other entities in the schema. A strong entity always has a primary key, and it is represented by a single rectangle in the ER diagram.



Weak Entity – Weak entity type doesn't have a key attribute and so we cannot uniquely identify them by their attributes alone. Therefore, a foreign key must be used in combination with its attributes to create a primary key. They are called Weak entity types because they can't be identified on their own. It relies on another powerful entity for its unique identity. A weak entity is represented by a double-outlined rectangle in ER diagrams.



Strong Entity v/s Weak Entity

Strong Entity	Weak Entity
Strong entity always has a primary key.	While a weak entity has a partial discriminator key.
Strong entity is not dependent on any other entity.	Weak entity depends on strong entity.
Strong entity is represented by a single rectangle.	Weak entity is represented by a double rectangle.

Strong Entity	Weak Entity
Two strong entity's relationship is represented by a single diamond.	While the relation between one strong and one weak entity is represented by a double diamond.
Strong entities have either total participation or not.	While weak entity always has total participation.

Relationships in DBMS

Definition

A relationship is defined as an association among several entities.

Relationship Set

A set of relationships of similar type is called a relationship set. Like entities, a relationship too can have attributes. These attributes are called descriptive attributes.

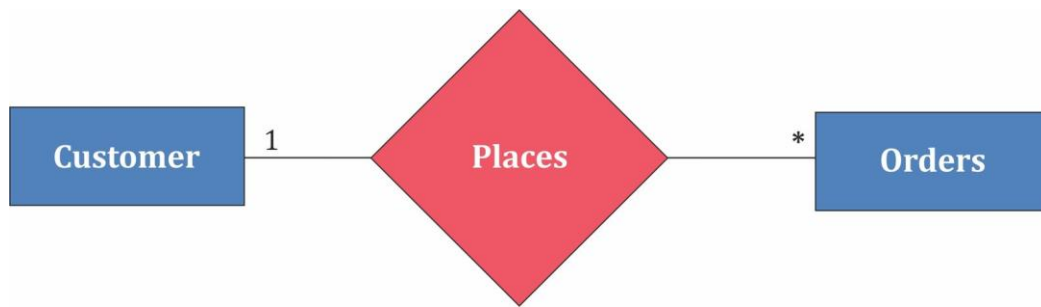
Mapping Cardinalities:

express the number of entities to which another entity can be associated via a relationship. For binary relationship sets between entity sets A and B, the mapping cardinality must be one of:

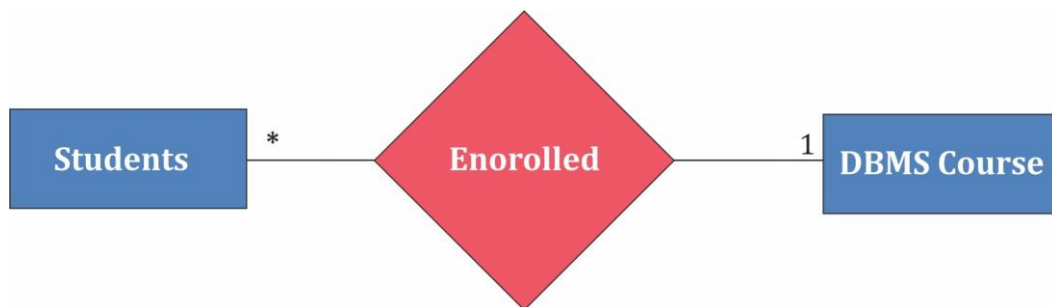
One-to-one: An entity in A is associated with at most one entity in B, and an entity in B is associated with at most one entity in A.



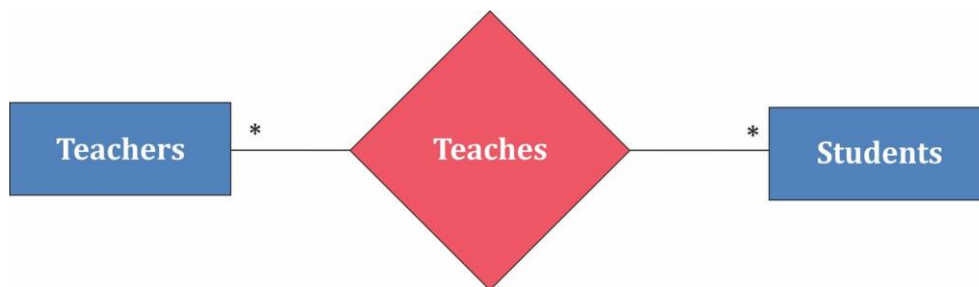
One-to-many: An entity in A is associated with any number in B. An entity in B is associated with at most one entity in A.



Many-to-one: An entity in A is associated with at most one entity in B. An entity in B is associated with any number in A.



Many-to-many: Entities in A and B are associated with any number from each other.



The appropriate mapping cardinality for a particular relationship set depends on the real world being modelled.

Degree of a Relationship Set

The number of entities sets that participate in a relationship set is termed as the degree of that relationship set. Thus,

Degree

Degree of a relationship set = Number of entities sets participating in a relationship set

Types of Relationship Sets

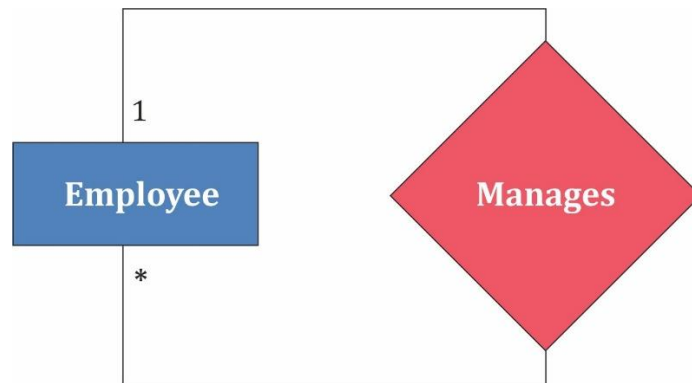
Based on degree of a relationship set, a relationship set can be classified into the following types-

- **Unary relationship set**
- **Binary relationship set**

- Ternary relationship set

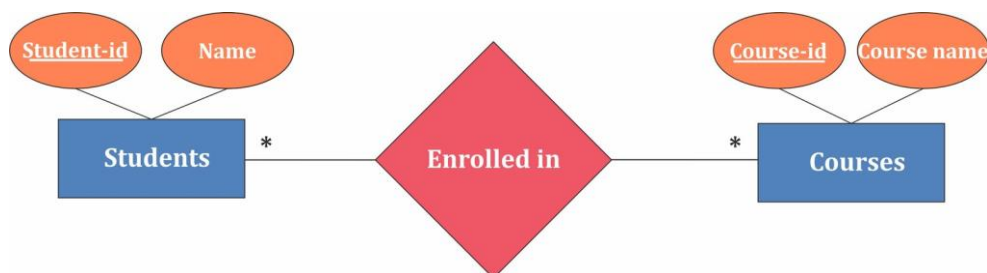
Unary Relationship Set

Unary relationship set is a relationship set where only one entity set participates in a relationship set.



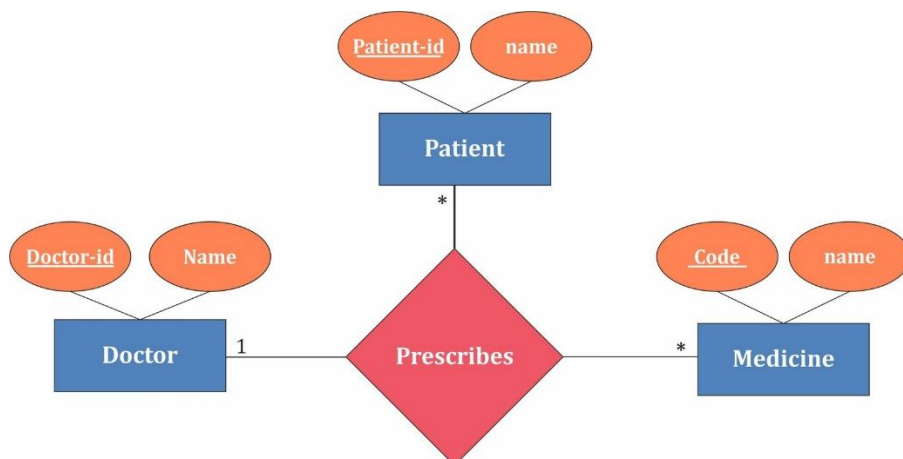
Binary Relationship Set

Binary relationship set is a relationship set where two entity sets participate in a relationship set.



Ternary Relationship Set

Ternary relationship set is a relationship set where three entity sets participate in a relationship set.



Participation Constraints

Total Participation of an Entity set

Total participation of an entity set represents that each entity in entity set must have at least one relationship in a relationship set. It is also called mandatory participation. Total participation is represented using a double line between the entity set and relationship set.

Partial participation of an Entity Set

Partial participation of an entity set represents that each entity in the entity set may or may not participate in the relationship instance in that relationship set. It is also called as optional participation. Partial participation is represented using a single line between the entity set and relationship set.

Additional Features of the ER Model in DBMS

As the complexity of **data** increased, it became more difficult to use the traditional ER Model for database modelling. Hence some Additional Features were made to the existing ER Model to make it able to handle the complex applications better.

Hence, three new concepts were added to the existing ER Model, they were:

- **Generalization**
- **Specialization**
- **Aggregation**

Some entities have relationships that form hierarchies. For instance, Employee can be an hourly employee or contracted employee.

In this relationship hierarchies, some entities can act as superclass and some other entities can act as subclass.

Superclass: An entity type that represents a general concept at a high level, is called superclass.

Subclass: An entity type that represents a specific concept at lower levels, is called subclass.

The subclass is said to inherit from superclass. When a subclass inherits from one or more superclasses, it inherits all their attributes. In addition to the inherited attributes, a subclass can also define its own specific attributes.

The symbol used for specialization/ Generalization is



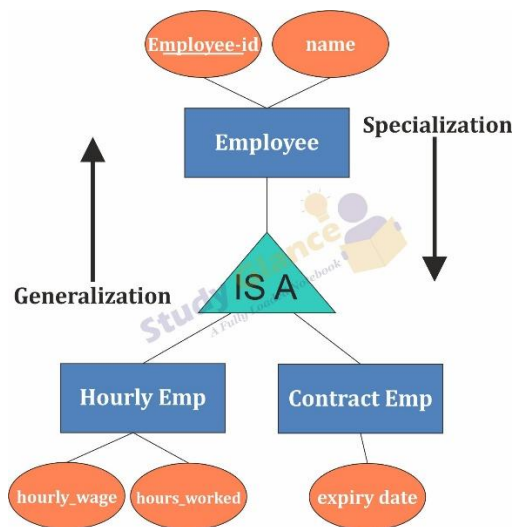
Generalization

Generalization is a process of extracting common properties from a set of entities and creating a generalized entity from it. It is a bottom-up approach, and it helps to reduce the size and complexity of the schema.

Example: Let us take two low-level entities as Car and Bus, and these two will have many common attributes and some specific attributes. And We will generalize and link the common attributes to the newly formed high-level entity named Vehicle.

Specialization

Specialization is opposite to Generalization. In this, entity is divided into subentities bases on their charactertics(distingvishing features). It breaks an entity into multiple entities from higher level to lower level. It is a top down approach.



Aggregration

Aggregation refers to the process by which entities are combined to form a single meaningful entity. The specific entities are combined because they do not make sense on their own. To establish a single entity, aggregation creates a relationship that combines these entities. The resulting entity makes sense because it enables the system to function well.